```
$ node breakfast.js --mode=async
```

JavaScript □同步

// □□□□□□□□ Promise □ async/await

sync
□隊等待

promise
未來值容器

await
同步的外表

event loop
幕後調度

```
$ cat menu.md
```

今日菜單

01 / □□□□

□□□□ vs □同步開工

02 / Promise

三種狀態、then/catch、逾時保護

03 / async & await

□□□□□□ vs 並行

04 / Event Loop

VIP 通道與執行順序測驗

目標：看懂任何一段 `async` 程式碼的「實際執行順序」。

PART 01

□ 餐比喻

□□□□□□□□□□□□

□□□□□□□□□□□

```
function makeCoffee() {
    console.log("□□□□□...");
    const start = Date.now();
    while (Date.now() - start < 2000) {} // □□□
    console.log("□□□□□");
}

function prepareBreakfastSync() {
    makeCoffee(); // 2s
    makeToast();  // 1.5s
    fryEgg();     // 1s
}
```

CONSOLE — 4.5s

□□□□□...
□□□□□
□□□□□...
□□□□□□
□□□□□□□□□□□□□

□□□□□□□□□□□

```javascript
function makeCoffee() {
    return new Promise((resolve) => {
        console.log("□□□□□...");
        setTimeout(() => {
            resolve("□□□□□");
        }, 2000); // □□□□□□
    });
}

async function prepareBreakfastAsync() {
    const coffee = await makeCoffee();
    console.log(coffee);
    // ...□□□□
}
```

CONSOLE — ~2s

□□□□□...
□□□□□...
□□□□...
□□□□□□□□□□□□□□□□

□□□□□□□□□□□□

親眼比一次

```
$ node breakfast.js
```

選一種方式開始做早餐

□□ vs 非同步：早餐賽跑

▶ □步（一件做完才做下一件）

▶ 非□步（全部□時開工）

☕ 煮咖啡（2s）

🍞 烤吐司（1.5s）

🍳 煎蛋（1s）

同步總時間 = 2 + 1.5 + 1 = **4.5** □；非同步總時間 = max(2, 1.5, 1) = **2** □

PART 02

Promise

□□□□□□□□□□□□□

三種狀態，單向變化

pending

待定：還在煮

fulfilled

已解決：resolve("▯▯")

rejected

已拒絕：reject("▯▯")

```javascript
let myPromise = new Promise((resolve, reject) => {
    let success = true;
    setTimeout(() => {
        if (success) resolve("▯▯▯");
        else reject("▯▯▯");
    }, 2000);
});

myPromise.then(result => console.log(result))
         .catch(error => console.log(error));
```

逾時保護：Promise.race + AbortController

```javascript
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const fetchWithTimeout = (url, { timeout = 3000 } = {}) => {
    const controller = new AbortController();
    const timer = delay(timeout).then(() => controller.abort());

    const request = fetch(url, { signal: controller.signal });

    return Promise.race([request, timer]).catch((error) => {
        if (error.name === "AbortError") return Promise.reject("請求逾時");
        return Promise.reject(error.message || "未知錯誤");
    });
};
```

Promise.race

[illegible] —— [illegible]

AbortController

真的取消請求，不浪費資源

PART 03

async / await

□□□□□□□□□□□□□

then NO GLYPH → await

```
async function loadSpells() {
    try {
        const res = await fetch("https://hp-api.onrender.com/api/spells");
        const data = await res.json();
        console.log("spells", data.length);
    } catch (err) {
        console.error(err.message);
    }
}
```

□□**□□□□**□await □□□ then □□□□□□□□□□□□□ □□□□ async/await□□□□□□ then□

串行：有相依才排隊

```javascript
async function serial() {
    const coffee = await makeCoffee();
    const toast = await makeToast();
    return [coffee, toast];
}
```

TOTAL TIME

≈ 所有任務時□總和（2 + 1.5 = 3.5s）

□用

□□ token □□ API —— 後者依賴前者

並行：沒相依就同開

```javascript
async function parallel() {
    const [coffee, toast] = await Promise.all([
        makeCoffee(),
        makeToast(),
    ]);
    return [coffee, toast];
}
```

TOTAL TIME

≈ 最慢□務（max(2, 1.5) = 2s）

□用

□□□□□□□□□ API

Promise.all vs allSettled

	`Promise.all`	`Promise.allSettled`
□□□□□	□□ reject □□□□	□□□□□□□
□□	□□□□	`{status, value/reason}` □□
□□	□□□□□□□	□□□□□□□□□□□

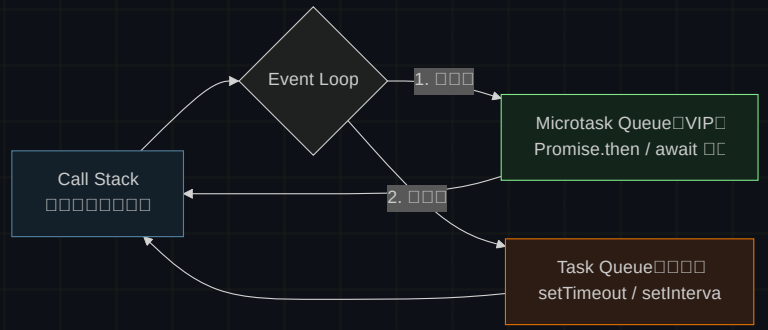
```
$ □□□□□all □□□□□□□□□□allSettled □□□□□□□□
```

PART 04

Event Loop

□□□□ VIP □□□□□□□□□□□

□□□□□□□□□□


Event Loop
1. □□□
Microtask Queue□VIP□
Promise.then / await □□
Call Stack
□□□□□□□□
2. □□□
Task Queue□□□□
setTimeout / setInterva

□□□□VIP□

□□□□□□□□□□□

□□□□□□□

□□□□□□□□□□□□□

執行順序測驗

```javascript
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

ANSWER

1 → 4（同步代碼先跑完，Call Stack 清空）
→ 3（微任務 VIP 通道優先）
→ 2（宏任務最後，即使延遲是 0）

□□□□□□1 → 4 → 3 → 2

[illegible]

[illegible]

[illegible]

Promise

[illegible]pending → fulfilled / rejected

[illegible] vs [illegible]

[illegible] await [illegible] Promise.all

Event Loop

[illegible]

[illegible] = [illegible]

Breakfast served.

□□□□fetch □ AbortController □□□□□□

```
$ node breakfast.js --mode=async
promise → await → race → event loop
```